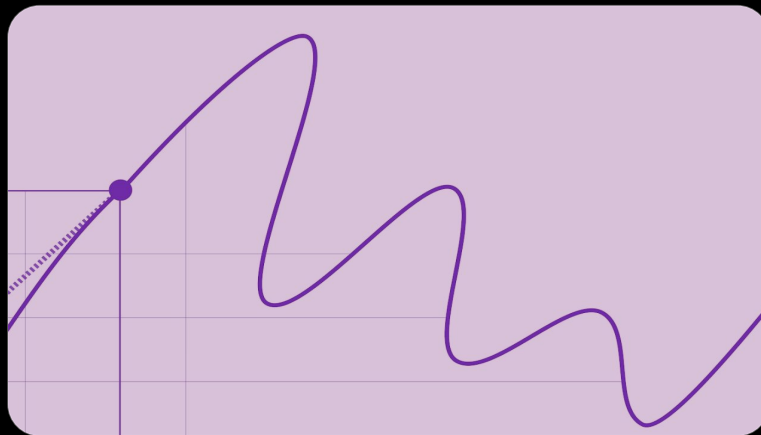
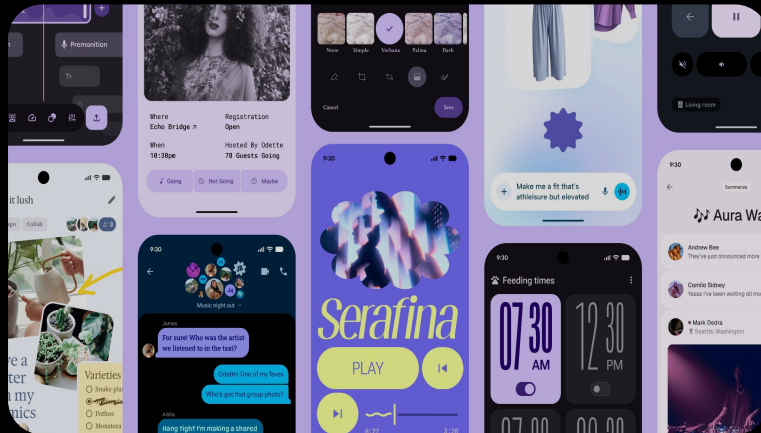


App Task Flow

Prof. Dr. Fernando Sambinelli
IFSP Hortolândia

Vamos Começar



App Task Flow

Crie e gerencie suas tarefas

- × Ideia do App: usuário cadastra e gerencia sua lista de tarefas



Dependências

Estrutura de Dependências

```
build.gradle.kts (app) ×
7 android {
35 }
36 kotlinOptions {
37     jvmTarget = "11"
38 }
39 buildFeatures {
40     compose = true
41 }
42 }
43
44 dependencies {
45     implementation(libs.androidx.core.ktx)
46     implementation(libs.androidx.lifecycle.runtime.ktx)
47     implementation(libs.androidx.activity.compose)
48     implementation(platform(dependencyProvider = libs.androidx.compose.bom))
49     implementation(libs.androidx.compose.ui)
50     implementation(libs.androidx.compose.ui.graphics)
51     implementation(libs.androidx.compose.ui.tooling.preview)
52     implementation(libs.androidx.compose.material3)
53     testImplementation(libs.junit)
54     androidTestImplementation(libs.androidx.junit)
55     androidTestImplementation(libs.androidx.espresso.core)
56     androidTestImplementation(platform(dependencyProvider = libs.androidx.compose.bom))
57     androidTestImplementation(libs.androidx.compose.ui.test.junit4)
58     debugImplementation(libs.androidx.compose.ui.tooling)
59     debugImplementation(libs.androidx.compose.ui.test.manifest)
60     // Lifecycle & ViewModel
61     implementation(libs.androidx.lifecycle.viewmodel.compose)
62     implementation(libs.androidx.lifecycle.runtime.compose)
63 }
```

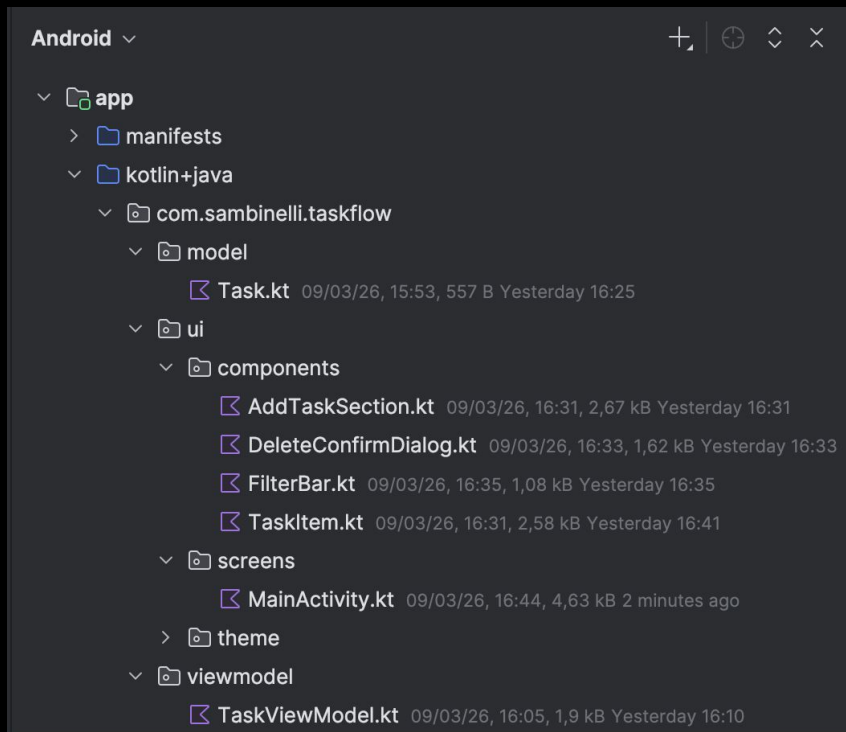
```
libs.versions.toml ×
1 [versions]
2 app = "8.13.2"
3 kotlin = "2.0.21"
4 coreKtx = "1.17.0"
5 junit = "4.13.2"
6 junitVersion = "1.3.0"
7 espressoCore = "3.7.0"
8 lifecycleRuntimeKtx = "2.10.0"
9 activityCompose = "1.12.4"
10 composeBom = "2024.02.00"
11 lifecycleViewmodelCompose = "2.8.6"
12 lifecycleRuntimeCompose = "2.8.6"
13
14 [libraries]
15 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
16 junit = { group = "junit", name = "junit", version.ref = "junit" }
17 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
18 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
19 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
20 androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
21 androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
22 androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
23 androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
24 androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
25 androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
26 androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
27 androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
28 androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
29 androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-compose", version.ref = "lifecycleViewmodelCompose" }
30 androidx-lifecycle-runtime-compose = { group = "androidx.lifecycle", name = "lifecycle-runtime-compose", version.ref = "lifecycleRuntimeCompose" }
31
32 [plugins]
33 android-application = { id = "com.android.application", version.ref = "app" }
34 kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
35 kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
```

Estrutura do Projeto

Pacotes

Organização das Pastas

Veja como ficará a estrutura de pacotes do projeto



Data Class

Entidade e Fake Objects

```
Task.kt x
1 package com.sambinelli.taskflow.model
2
3 data class Task( 13 Usages
4     val id: Int,
5     val titulo: String,
6     val descricao: String,
7     val estaConcluido: Boolean = false
8 )
9
10 // fake objects
11 val todo1 = Task( 4 Usages
12     id = 1,
13     titulo = "Todo 1",
14     descricao = "Descricao para Tarefa 1",
15     estaConcluido = false,
16 )
17
18 val todo2 = Task(
19     id = 2,
20     titulo = "Todo 2",
21     descricao = "Descricao para Tarefa 2",
22     estaConcluido = true,
23 )
24
25 val todo3 = Task(
26     id = 3,
27     titulo = "Todo 3",
28     descricao = "Descricao para Tarefa 3",
29     estaConcluido = false,
30 )
```

ViewModel

Classe TaskViewModel

ViewModel

Ele recebe os eventos da UI, processa a lógica de negócio e atualiza os estados. Esses estados atualizados são então retornados à interface, que se recompõe a UI.

```
TaskViewModel.kt
class TaskViewModel : ViewModel() { 2 Usages

    // Estado da lista de tarefas
    private val _tasks = mutableStateListOf<Task>() 7 Usages

    val tasks: List<Task> get() = _tasks // val tasks: List<Task> get() {return _tasks} 3 Usages

    // Controle do filtro ativo
    var activeFilter by mutableStateOf<Filter>{ Filter.ALL } 3 Usages
    private set

    // Tarefa selecionada para exclusão (abre o dialog)
    var taskToDelete by mutableStateOf<Task?>{ null } 6 Usages
    private set

    // Contador para gerar IDs únicos
    private var nextId = 1 1 Usage

    // Lista filtrada derivada do estado principal
    val filteredTasks: List<Task> 2 Usages
    get() = when (activeFilter) {
        Filter.ALL -> tasks
        Filter.PENDING -> tasks.filter { !it.estaConcluido }
        Filter.DONE -> tasks.filter { it.estaConcluido }
    }

    fun addTask(titulo: String, descricao: String) { 1 Usage
        if (titulo.isBlank()) return
        _tasks.add(Task(id = nextId++, titulo = titulo, descricao = descricao))
    }

    fun toggleDone(task: Task) { 1 Usage
        val index = _tasks.indexOfFirst { it.id == task.id }
        if (index != -1) {
            _tasks[index] = _tasks[index].copy(estaConcluido = !_tasks[index].estaConcluido)
        }
    }

    fun requestDelete(task: Task) { 1 Usage
        taskToDelete = task
    }

    fun confirmDelete() { 1 Usage
        taskToDelete?.let { _tasks.removeIf { t -> t.id == it.id } }
        taskToDelete = null
    }

    fun cancelDelete() { 1 Usage
        taskToDelete = null
    }

    fun setFilter(filter: Filter) { 1 Usage
        activeFilter = filter
    }
}

enum class Filter { ALL, PENDING, DONE } 12 Usages
```

AddTaskSection

Componente da UI

PreviewAddTask

Nova Tarefa

Título *

Descrição (opcional)

+ Adicionar

```
24 @Composable 3 Usages
25 fun AddTaskSection(
26     titulo: String,
27     descricao: String,
28     onTitleChange: (String) -> Unit,
29     onDescriptionChange: (String) -> Unit,
30     onAddClick: () -> Unit
31 ) {
32     Card(
33         modifier = Modifier.fillMaxWidth(),
34         colors = CardDefaults.cardColors(
35             containerColor = MaterialTheme.colorScheme.surfaceVariant
36         )
37     ) {
38         Column(modifier = Modifier.padding( all = 16.dp)) {
39
40             Text(text = "Nova Tarefa", style = MaterialTheme.typography.titleMedium)
41
42             Spacer(modifier = Modifier.height( height = 8.dp))
43
44             OutlinedTextField(
45                 value = titulo,
46                 onValueChange = onTitleChange,
47                 label = { Text(text = "Título *") },
48                 modifier = Modifier.fillMaxWidth(),
49                 singleLine = true
50             )
51
52             Spacer(modifier = Modifier.height( height = 8.dp))
53
54             OutlinedTextField(
55                 value = descricao,
56                 onValueChange = onDescriptionChange,
57                 label = { Text(text = "Descrição (opcional)") },
58                 modifier = Modifier.fillMaxWidth(),
59                 maxLines = 2
60             )
61
62             Spacer(modifier = Modifier.height( height = 12.dp))
63
64             Button(
65                 onClick = onAddClick,
66                 modifier = Modifier.align(Alignment.End),
67                 enabled = titulo.isNotBlank()
68             ) {
69                 Icon(imageVector = Icons.Default.Add, contentDescription = null)
70                 Spacer(modifier = Modifier.width( width = 4.dp))
71                 Text(text = "Adicionar")
72             }
73         }
74     }
75 }
76
77 @Preview
78 @Composable
79 fun PreviewAddTask() {
80     AddTaskSection(titulo="", descricao="", onTitleChange = {}, onDescriptionChange = {}, onAddClick = {})
81 }
```

TaskItem

Componente da UI

PreviewTaskItem



Todo 1

Descricao para Tarefa 1



```
TaskItem.kt x
@Composable 3 Usages
fun TaskItem(
    task: Task,
    onToggleDone: () -> Unit,
    onDeleteRequest: () -> Unit
) {
    Card(modifier = Modifier.fillMaxWidth()) {
        Row(
            modifier = Modifier
                .padding(all = 12.dp)
                .fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Checkbox(
                checked = task.estaConcluido,
                onCheckedChange = { onToggleDone() }
            )

            Column(modifier = Modifier.weight(weight = 1f)) {
                Text(
                    text = task.titulo,
                    style = MaterialTheme.typography.bodyLarge,
                    textDecoration = if (task.estaConcluido) TextDecoration.LineThrough else null,
                    color = if (task.estaConcluido) MaterialTheme.colorScheme.outline
                        else MaterialTheme.colorScheme.onSurface
                )
                if (task.descricao.isNotBlank()) {
                    Text(
                        text = task.descricao,
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.outline
                    )
                }
            }

            IconButton(onClick = onDeleteRequest) {
                Icon(
                    imageVector = Icons.Default.Delete,
                    contentDescription = "Excluir tarefa",
                    tint = MaterialTheme.colorScheme.error
                )
            }
        }
    }
}

@Preview
@Composable
fun PreviewTaskItem() {
    TaskItem(task = todo1, onToggleDone={}, onDeleteRequest={})
}
```

DeleteConfirm Dialog

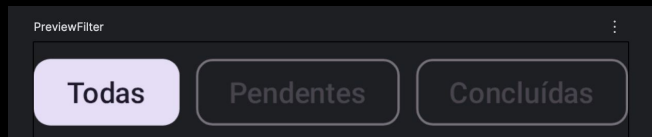
Componente da UI



```
16  @Composable 3 Usages
17  fun DeleteConfirmDialog(
18      taskTitle: String,
19      onConfirm: () -> Unit,
20      onDismiss: () -> Unit
21  ) {
22      AlertDialog(
23          onDismissRequest = onDismiss,
24          icon = { Icon(imageVector = Icons.Default.Warning, contentDescription = null) },
25          title = { Text(text = "Excluir tarefa?") },
26          text = {
27              Text(text = "A tarefa \"$taskTitle\" será removida permanentemente. Deseja continuar?")
28              //\" é para exibir o \" na mensagem
29          },
30          confirmButton = {
31              Button(
32                  onClick = onConfirm,
33                  colors = ButtonDefaults.buttonColors(
34                      containerColor = MaterialTheme.colorScheme.error
35                  )
36              ) {
37                  Text(text = "Excluir")
38              }
39          },
40          dismissButton = {
41              OutlinedButton(onClick = onDismiss) {
42                  Text(text = "Cancelar")
43              }
44          }
45      )
46  }
47
48  @Preview
49  @Composable
50  fun PreviewDialog(){
51      DeleteConfirmDialog(taskTitle = todo1.titulo, onConfirm={}, onDismiss={})
52  }
```

FilterBar

Componente da UI



```
FilterBar.kt x
12  @Composable 3 Usages
13  fun FilterBar(activeFilter: Filter, onFilterSelected: (Filter) -> Unit) {
14      Row(
15          horizontalArrangement = Arrangement
16              .spacedBy( space = 8.dp)
17      ) {
18          Filter.entries.forEach { filter ->
19              FilterChip(
20                  selected = activeFilter == filter,
21                  onClick = { onFilterSelected(filter) },
22                  label = {
23                      Text(
24                          text = when (filter) {
25                              Filter.ALL -> "Todas"
26                              Filter.PENDING -> "Pendentes"
27                              Filter.DONE -> "Concluídas"
28                          }
29                      )
30                  }
31              )
32          }
33      }
34  }
35  @Composable
36  @Preview
37  fun PreviewFilter(){
38      FilterBar( activeFilter = Filter.ALL , onFilterSelected = {})
39  }
```

Main Activity

Estrutura da tela (1/2)

```
37 ></> class MainActivity : AppCompatActivity() { 1 Usage
38     override fun onCreate(savedInstanceState: Bundle?) {
39         super.onCreate(savedInstanceState)
40         enableEdgeToEdge()
41         setContent {
42             TaskFlowTheme {
43                 TaskScreen()
44             }
45         }
46     }
47 }
48
49 @OptIn(markerClass = ExperimentalMaterial3Api::class) 2 Usages
50 @Composable
51 fun TaskScreen(viewModel: TaskViewModel = viewModel()) {
52
53     var titleInput by remember { mutableStateOf(value = "") }
54     var descriptionInput by remember { mutableStateOf(value = "") }
55
56     // Dialog de confirmação de exclusão
57     if (viewModel.taskToDelete != null) {
58         DeleteConfirmDialog(
59             taskTitle = viewModel.taskToDelete!!.titulo,
60             onConfirm = { viewModel.confirmDelete() },
61             onDismiss = { viewModel.cancelDelete() }
62         )
63     }
64
65     Scaffold(
66         topBar = {
67             TopAppBar(
68                 title = { Text(text = "TaskFlow") },
69                 colors = TopAppBarDefaults.topAppBarColors(
70                     containerColor = MaterialTheme.colorScheme.primaryContainer
71                 )
72             )
73         }
74     ) { padding ->
```

```
72
73     }
74 } { padding ->
75
76     Column(
77         modifier = Modifier
78             .fillMaxSize()
79             .padding(paddingValues = padding)
80             .padding(all = 16.dp)
81     ) {
82         // Seção de adição de tarefa
83         AddTaskSection(
84             titulo = titleInput,
85             descricao = descriptionInput,
86             onTitleChange = { titleInput = it },
87             onDescriptionChange = { descriptionInput = it },
88             onAddClick = {
89                 viewModel.addTask(titulo = titleInput, descricao = descriptionInput)
90                 titleInput = ""
91                 descriptionInput = ""
92             }
93         )
94
95         Spacer(modifier = Modifier.height(height = 16.dp))
96
97         // Barra de filtros
98         FilterBar(
99             activeFilter = viewModel.activeFilter,
100             onFilterSelected = { viewModel.setFilter(it) }
101         )
102
103         Spacer(modifier = Modifier.height(height = 8.dp))
104     }
105 }
```

Main Activity

Estrutura da tela (2/2)

```
104
105 // Contador de tarefas
106 Text(
107     text = "${viewModel.filteredTasks.size} tarefa(s) exibida(s)",
108     style = MaterialTheme.typography.labelMedium,
109     color = MaterialTheme.colorScheme.outline
110 )
111
112 Spacer(modifier = Modifier.height( height = 8.dp))
113
114 // Lista dinâmica de tarefas
115 LazyColumn(
116     verticalArrangement = Arrangement
117         .spacedBy( space = 8.dp)) {
118     items( items = viewModel.filteredTasks, key = { it.id }) { task ->
119         TaskItem(
120             task = task,
121             onToggleDone = { viewModel.toggleDone(task) },
122             onDeleteRequest = { viewModel.requestDelete(task) }
123         )
124     }
125 }
126
127 }
128
129
130 @Preview(showBackground = true)
131 @Composable
132 fun AppPreview() {
133     TaskFlowTheme {
134         TaskScreen()
135     }
136 }
```



Fernando Sambinelli

Professor e Doutor em Tecnologia
sambinelli@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo

